UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

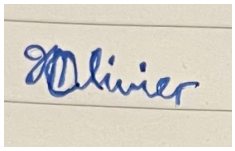
EPE 321

Software Engineering

Design Document

Group 11

By submitting this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

Name and Surname	Student Number	Signature	% contribution
Daniel Kamener	22646494		16.6
Nicole Phairah	21603384		16.6
Jarryd Olivier	23567873		16.6
Tshegofatso Phetlhe	23669391		16.6
Somtochi Ezebuike	21644502		16.6
Sean De Witt	23548917		16.6

Contents

I. Introduction

This document describes the final group design for the Job Swipe app. This system aims to make the application and hiring process more straightforward with an intuitive swipe-based format. Job seekers can create profiles, upload resumes, and swipe right to apply - while employers promote listings and manage applicants from a central dashboard. Communication between users is handled by an encrypted role-based messaging system where employers initiate conversations and jobseekers can reply. The design tasks were assigned to group members in a way that each task covered all necessary elements. Each of the following sections within this document maps to an individual deliverables outlined in the project requirements: UML class diagram, UML activity diagram, unit test tables, GUI models, and product and sprint backlogs. The design of all the artefacts has been performed at a low level to reduce ambiguity at the implementation and integration stages across different modules of the system.

In particular, great attention has been given to the Networking, Security, and Communication Layer - as secure authentication, user data protection, and reliable real-time communications are of paramount importance. The design involves using JWT for auth, password hashing, encrypting sensitive information, in-app and email notifications. Performance, usability and maintainability as specified in the SRS have also been taken into account.

The sections that follow aim to organise the description into layers of the system design for both functional aspects— such as user registration, job listing, and messaging—and non-functional requirements such as the security, scalability and the responsiveness of the app as a whole. The goal of these specifications are to establish a clear roadmap for the subsequent implementation phase and provides assurance that the final system will meet the specified requirements.

II. Overview and Team Responsibilities

JobSeekr is a web application that helps connect job seekers and employers. It follows a dating-app style format, which makes the user experience very simple. Job seekers simply swipe on the jobs they want to apply for, and the application will automatically be submitted for them. Employers can create one or multiple job listings and can view and manage all applications. JobSeekr streamlines the hiring process for both job seekers and employers, saving time while improving the overall experience of finding and filling jobs.

To guarantee the project's success, each group member was given specific tasks to complete. Daniel was given the role of team leader and will focus on front-end design, making the screens and user interface that recruiters and candidates utilize. Nicole is in charge of the back-end job application procedure, which involves avoiding duplicate application creation and adding resumes and profiles. Jarryd is in charge of back end and manages the job listings for recruiters, providing functionality to add, edit, and remove job posts. Somto creates and manages the database and server, constructs the API, and makes sure that data is efficiently and safely stored.

Tshego handles the AI aspect of the system and works on the Help Bot, which answers user questions and guides applicants through the system. Sean is responsible for system security, including user authentication, notifications, and data protection. Together, these roles make sure the system's front-end, back-end, database, security, and support are all built and work well together

III. UML Design

A. Frontend UML:

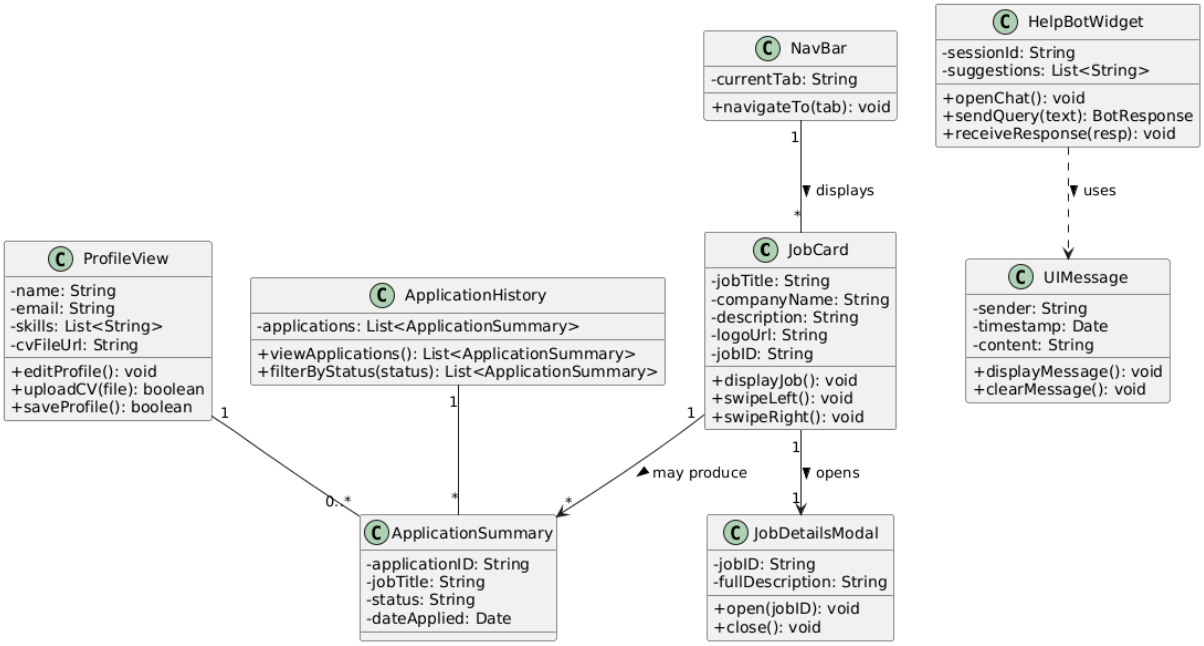


Figure 1: Frontend UML

B. Server UML

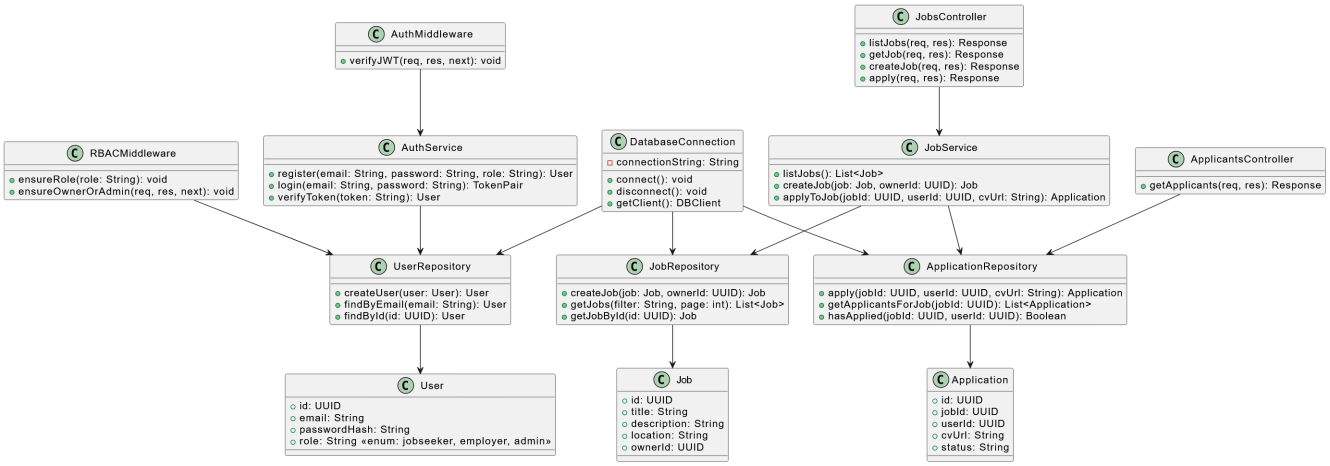


Figure 2: Server UML

C. Backend UML: Application

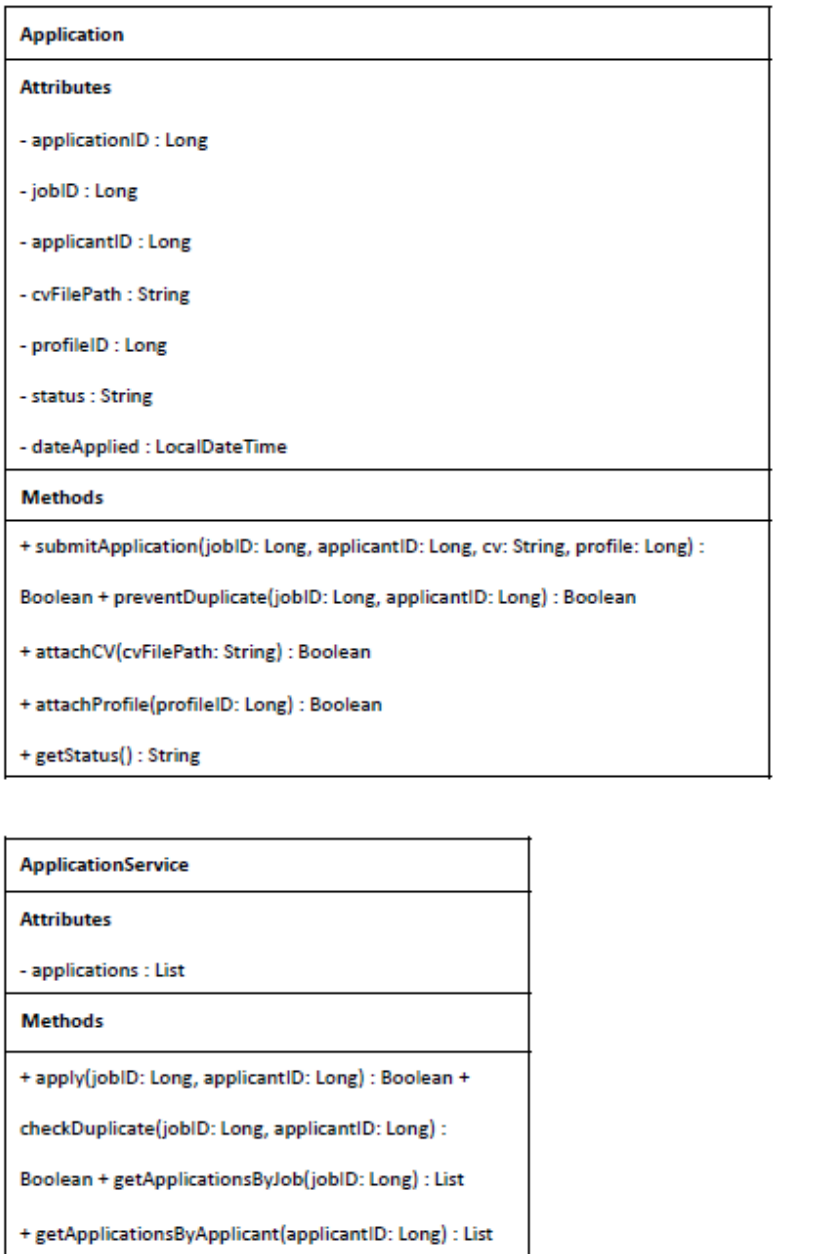


Figure 3: Backend UML: Job application

D. Backend UML: Employer

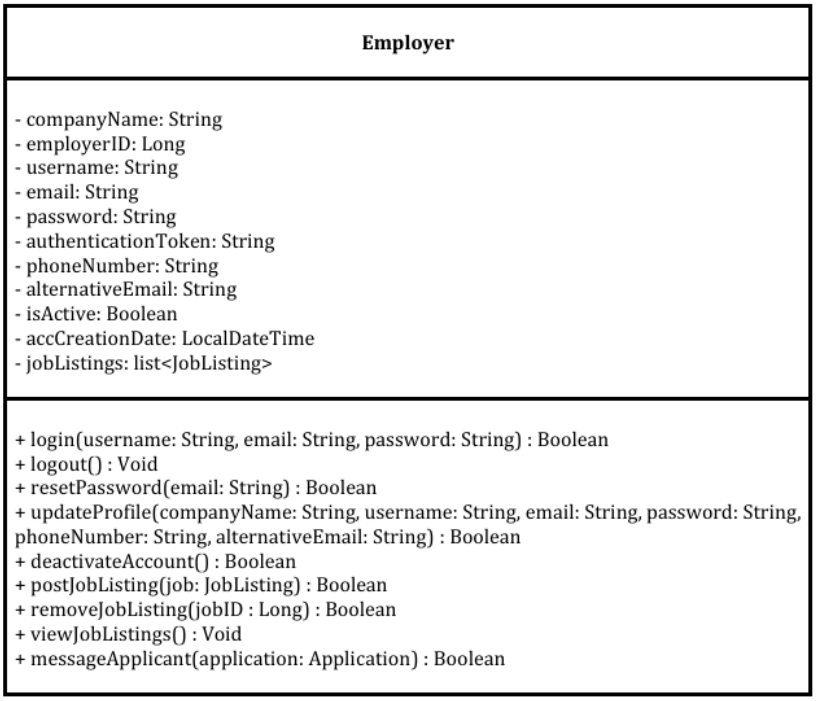


Figure 4: Backend UML: Employer User

E. Backend UML: Job Listings

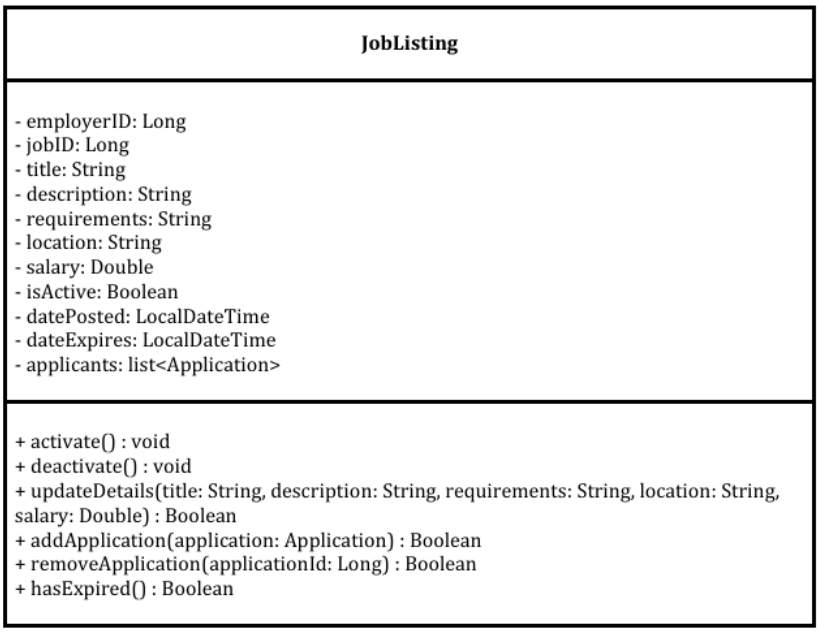


Figure 5: Backend UML: Job listings

F. Backend UML: Job Listing Manager

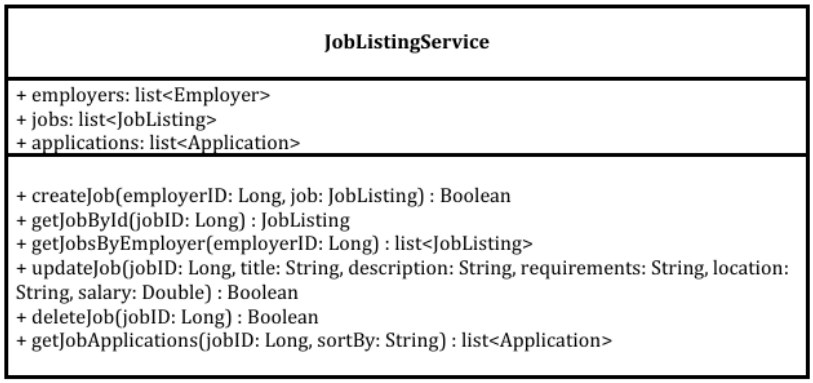


Figure 6: Backend UML: Job listing service

G. AI Help Bot UML

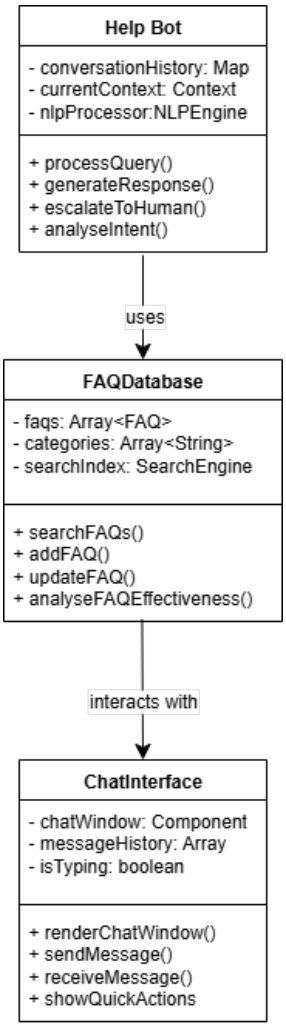


Figure 7: Help Bot UML

H. Backend UML: Authentication Service

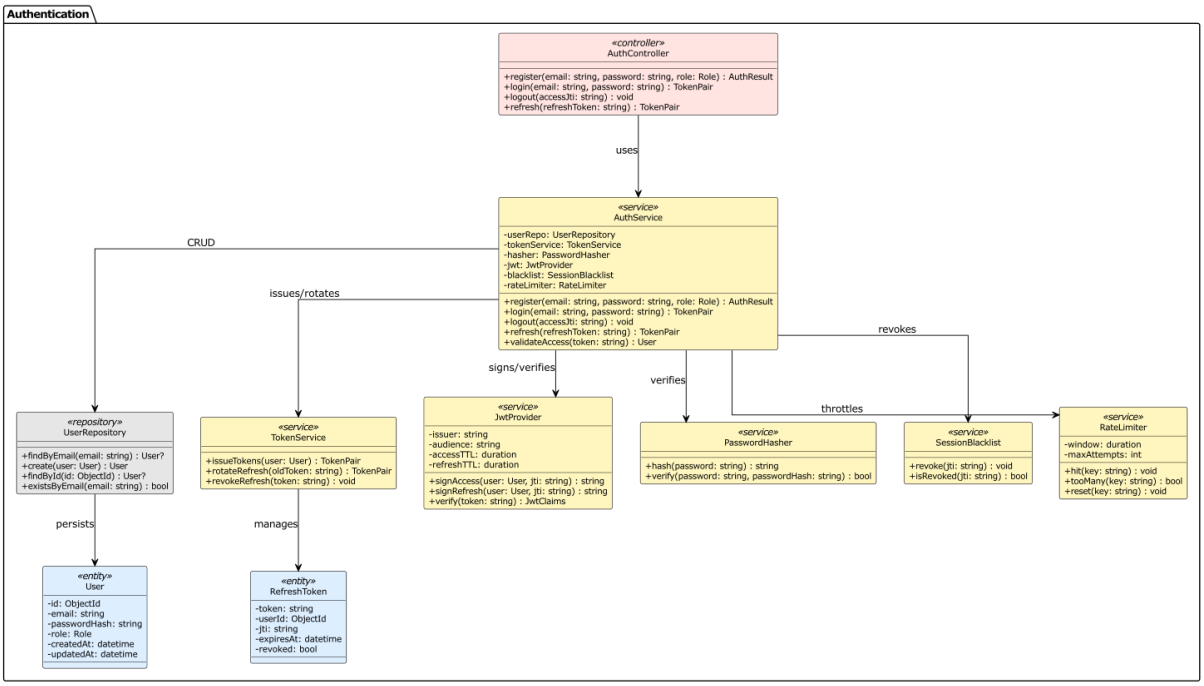


Figure 8: UML Class Diagram — Authentication Service

I. Backend UML: Messaging & Notification Service

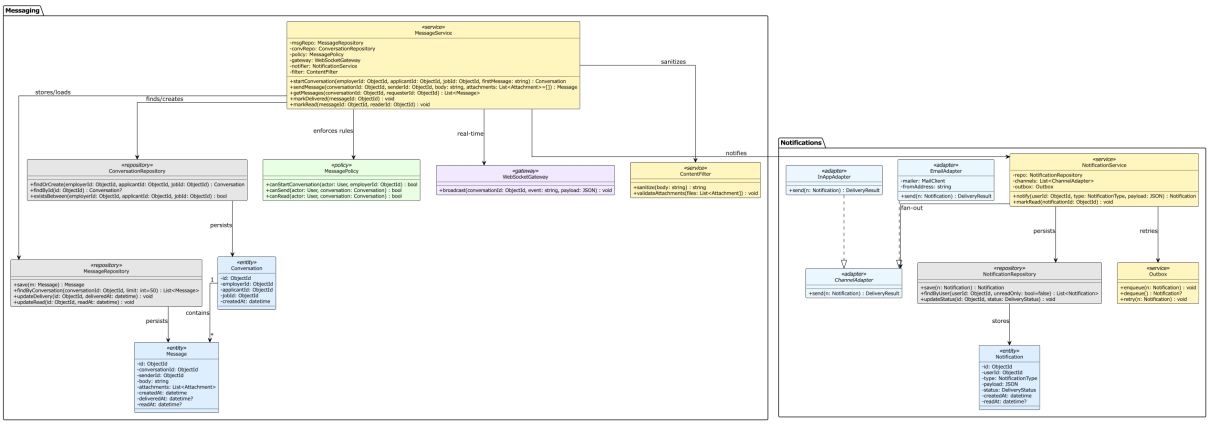


Figure 9: UML Class Diagram — Messaging & Notification Service

IV. Activity Diagrams

A. *Job Feed*

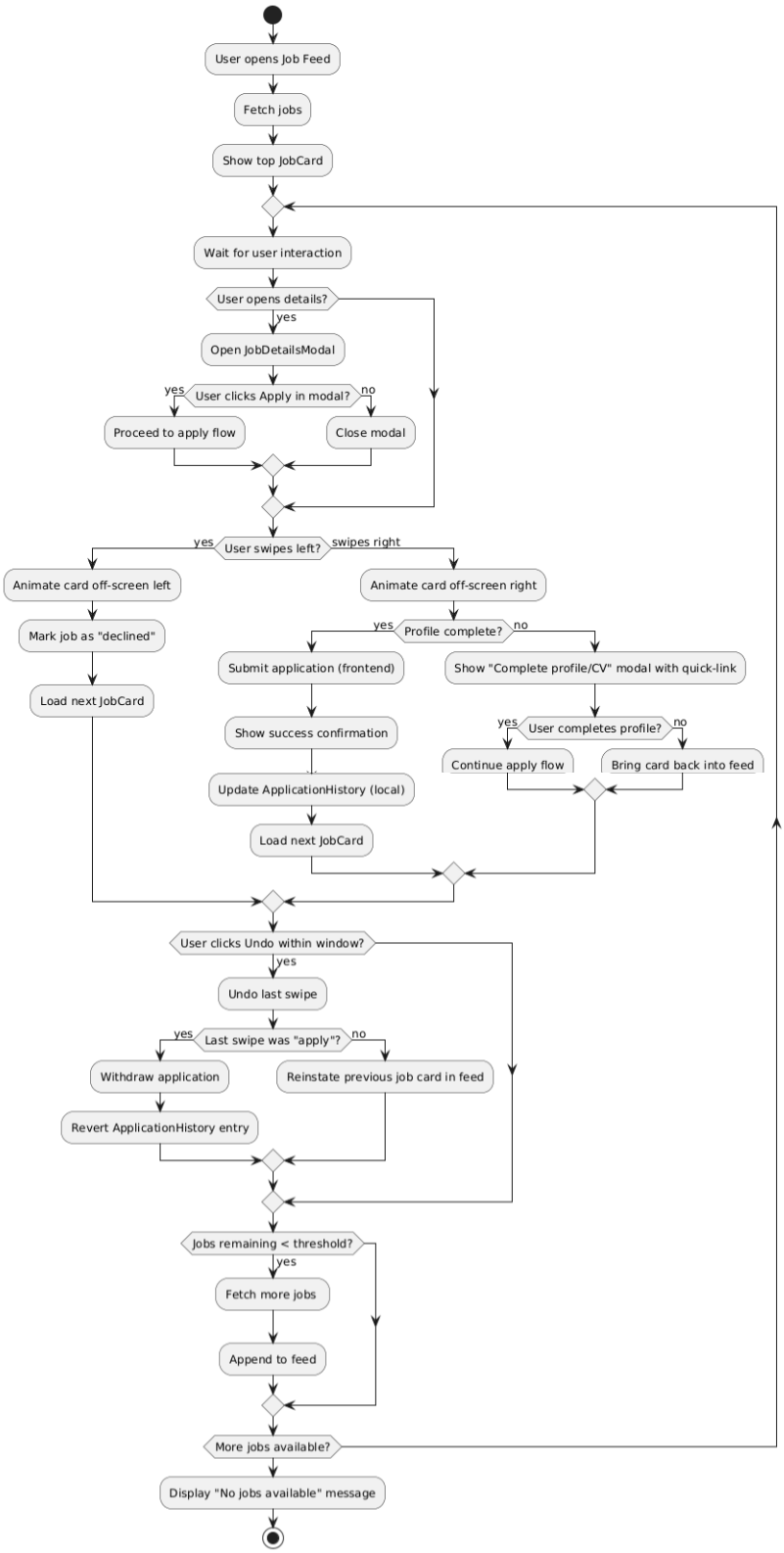


Figure 10: Job Feed Activity Diagram

B. Swipe Right

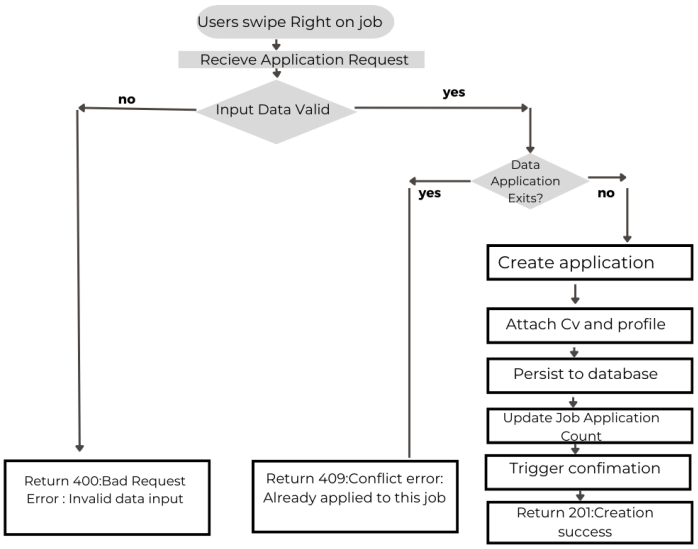


Figure 11: Swipe Right Application Diagram

C. Server

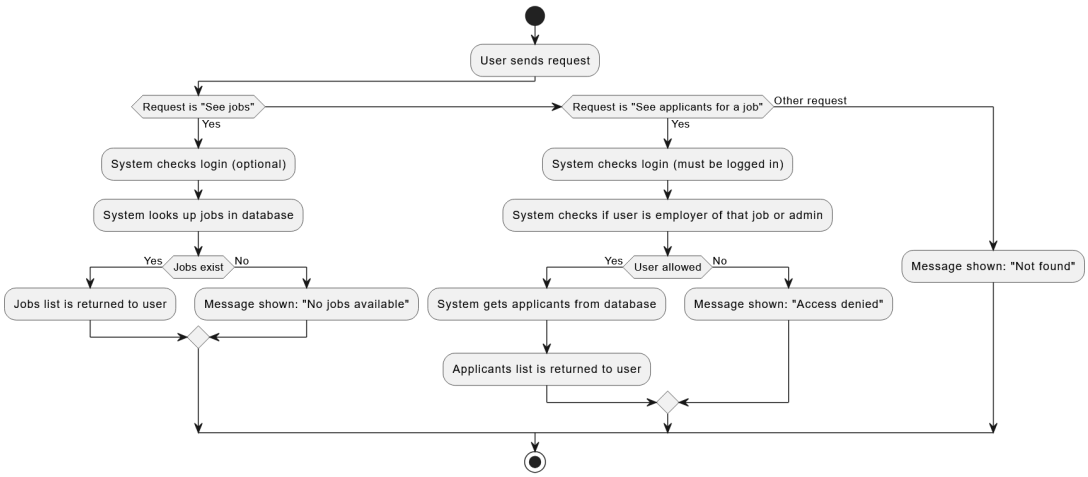


Figure 12: Server Activity Diagram

D. Employer

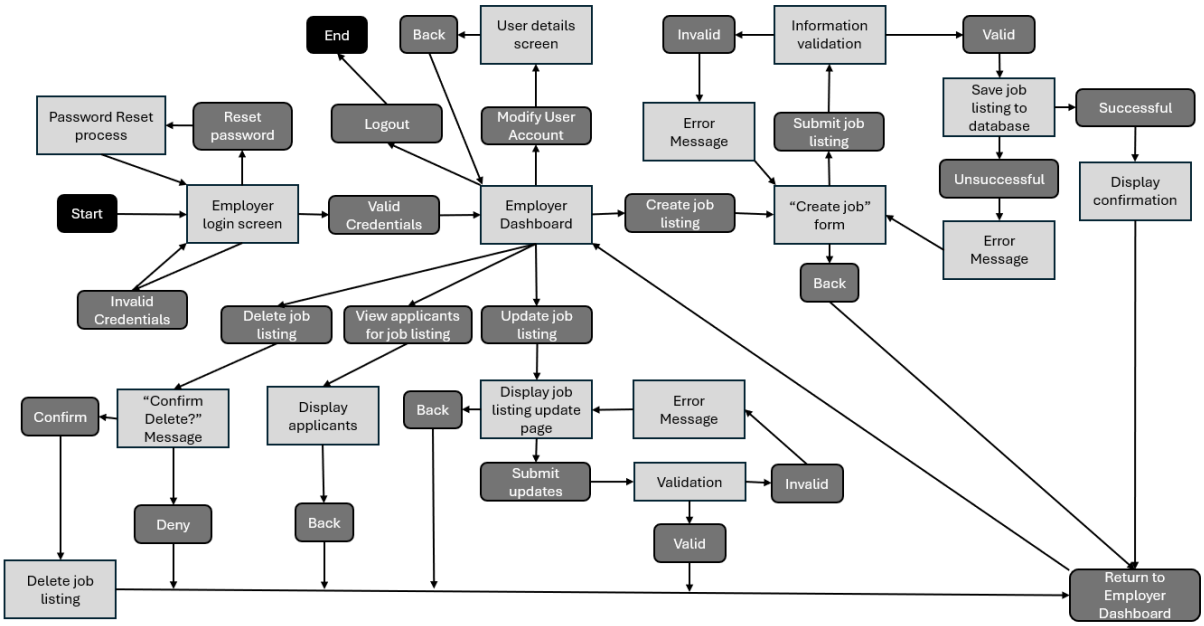


Figure 13: Employer user activity diagram

E. User Query

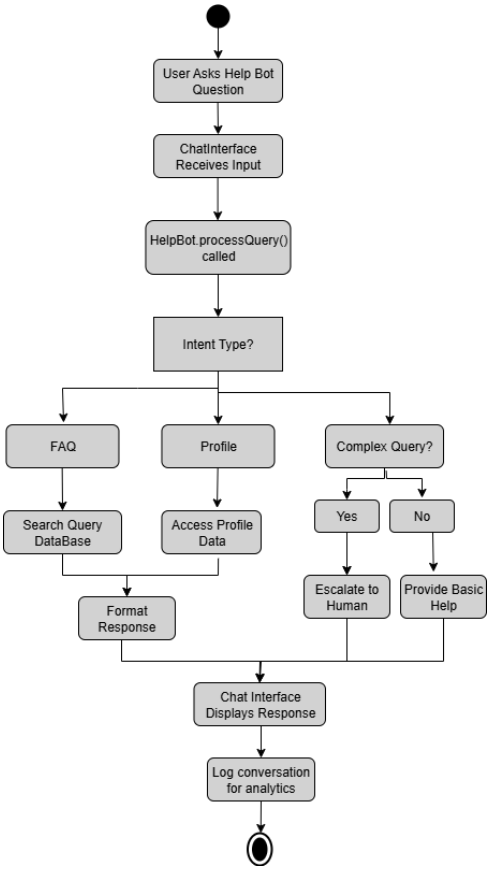


Figure 14: User Query Processor activity diagram

F. Employer Messages

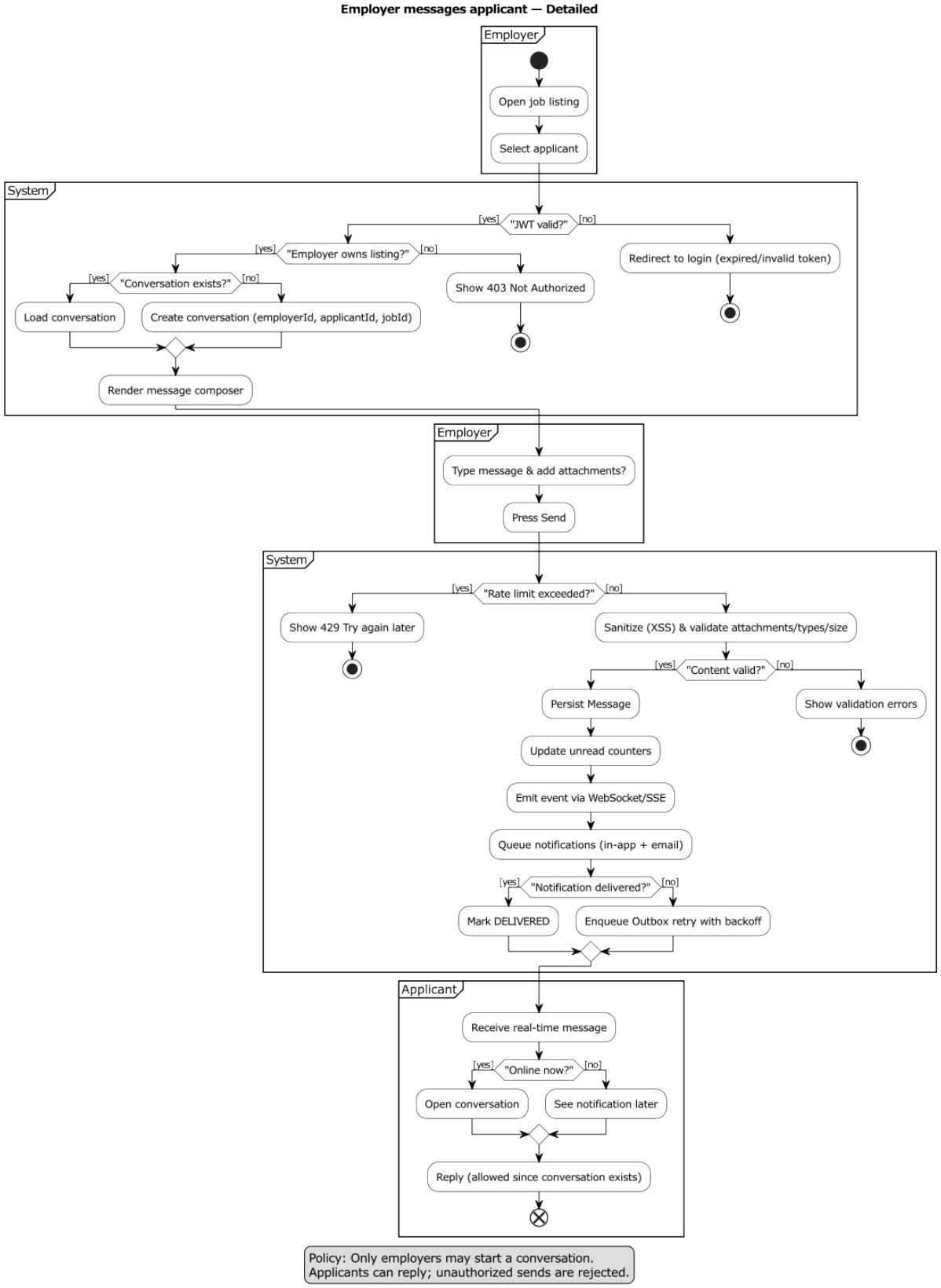


Figure 15: Employer Messages Applicant activity diagram

V. Unit Testing

A. Frontend

#	Component	Inputs	Expected Outcome
FE1	ProfileView	Enter invalid email	Error message "Please enter a valid email address"
FE2	ProfileView	Leave name field empty	Error "Name is required"
FE3	CVUploader	Upload incorrect attachment	Upload blocked and error "Unsupported file type"
FE4	CVUploader	Upload valid PDF	Progress bar updates and success shown
FE5	NavBar	Click to show Applications	Correct screen loaded and active tab updated
FE6	JobCard	Show with job details	Card shows job title, company, description, and logo
FE7	JobCard	Click "View Details"	Job Details Modal opens with job description and Apply button
FE8	JobFeed	Swipe left on job	Card removed and job not applied for
FE9	JobFeed	Swipe right with valid profile	application submitted and user shown "Application submitted"
FE10	JobFeed	Swipe right with incomplete profile	Error showing "Complete profile before applying"
FE11	JobFeed	Swipe right but fails	Error message "Application failed. Retry?" and retry button available
FE12	JobFeed	Swipe right success but user selects Undo	Brings card back and removes application from database
FE13	JobDetailsModal	Click Apply inside job details modal	application submitted and user shown "Application submitted". Modal closes after

TABLE I: Front-End Unit Tests: UI validation, navigation, and interactions

B. Backend-Application

#	Tested Function	Inputs	Expected Output
BE1	submitApplication	Valid jobID, applicantID, CV file path, profileID	The application is created with the correct information. Returns True
BE2	submitApplication	Invalid jobID/applicantID, missing profile, or missing CV	Application is not created. Returns False
BE3	preventDuplicate	Duplicate jobID and applicantID found	Returns True. A record already exists with the same jobID and applicantID
BE4	preventDuplicate	Unique jobID and applicantID	Returns False. Submission allowed
BE5	attachCV	Valid CV file path	CV is linked to application. Returns True
BE6	attachCV	Invalid CV path (empty or wrong format)	CV is not attached. Returns False
BE7	attachProfile	Valid profileID	Profile is linked to application. Returns True
BE8	attachProfile	Invalid or missing profileID	Profile is not attached. Returns False
BE9	getStatus	None	Returns the application's current status, e.g., "Pending," "Accepted," or "Declined"
BE10	apply (via ApplicationService)	Valid jobID and applicantID	The service list now includes the application. Returns True

TABLE II: Back-End Unit Tests: Application

C. Server

#	Component	Inputs	Expected Outcome
SE1	JobsController	Request job list when jobs exist	Jobs list is returned
SE2	JobsController	Request job list when no jobs exist	Empty list is returned with message
SE3	JobsController	Request job list but database fails	Error message shown
SE4	ApplicantsController	Employer asks for applicants of own job	List of applicants is returned
SE5	ApplicantsController	Employer asks for applicants of another job	Access denied message
SE6	ApplicantsController	Admin asks for applicants	List of applicants is returned
SE7	ApplicantsController	Ask for applicants of a job that does not exist	Job not found message
SE8	AuthMiddleware	Request without login token	Not allowed message
SE9	AuthMiddleware	Request with invalid token	Not allowed message
SE10	ApplicationRepository	Same user applies twice to same job	Duplicate application error

TABLE III: Server Unit Tests: API requests, responses, and errors

D. Backend:Employer

#	Tested Function	Inputs	Expected Output
1	login	Email or username and password of employer account	Successful login (True) if inputs match an account in the database, otherwise, unsuccessful login (False)
2	logout	None	Employer is logged out of their account
3	resetPassword	Email	Password variable is changed if steps were followed in received email
4	updateProfile	Data of profile fields to be changed	A change in the relevant account details that are changed to the inputted values. True if the inputs are valid and False if not
5	deactivateAccount	None	Deletion of the employer's account
6	postJobListing	JobListing object created with the job listing survey	The inputted jobListing is added to the Employer's jobListings list
7	removeJobListing	The jobID of the job that needs to be removed	True if the job was successfully removed, false if it was not
8	viewJobListings	None	Show the display for all of this Employer's jobListings
9	messageApplicant	The Application object for the applicant that needs to be messaged	Show the display where the Employer can message the specific applicant. True if the applicant was found, False if not

TABLE IV: Backend Unit Tests: Employer

E. Backend: JobListing

#	Tested Function	Inputs	Expected Output
1	activate	None	isActive = True
2	deactivate	None	isActive = False
3	updateDetails	Data of detail fields to be changed	A change in the relevant details that are changed to the inputted values. True if the inputs are valid and False if not
4	addApplication	Application to be added	Application is successfully added to applicants list (True). False if it was not added successfully.
5	removeApplication	The applicationID of the application that needs to be removed / rejected	The application is successfully removed from the applicants list (True). False if the removal was unsuccessful
6	hasExpired	None	True if the current jobListing is no longer accepting applicants, False if it is accepting applicants

TABLE V: Backend Unit Tests: JobListing

F. Backend Unit Tests: Networking, Security, and Communication Layer

#	**Component**	**Inputs**	**Expected Output**
A1	AuthService.register	email=new@example.com; password=Strong1!; role=Employer	201 Created; user persisted; password hashed (not encrypted)
A2	AuthService.register (duplicate)	email=existing@example.com; password=Any	409 Conflict; no user created
A3	AuthService.login	email=valid@example.com; password=correct	TokenPair{access, refresh}; access JWT has sub, role, iat, exp; refresh persisted
A4	AuthService.login (invalid pw)	email=valid@example.com; password=wrong	401 Unauthorized; rateLimiter.hit(key) called
A5	AuthService.login (unknown email)	email=none@example.com; password=any	401 Unauthorized
A6	AuthService.refresh	refresh=valid_and_not_revoked	New TokenPair issued; old revoked; new jti stored
A7	AuthService.refresh (revoked)	refresh=revoked_or_used	401 Unauthorized
A8	AuthService.refresh (expired)	refresh=expired	401 Unauthorized
A9	AuthService.logout	accessJti=jti123	blacklist.revoke(jti123); later verify revoked/401
A10	JwtProvider.verify	token=tampered_signature	Signature error thrown
A11	PasswordHasher	password=P@ssw0rd!	Hash != password; verify(password, hash) == true
A12	RateLimiter	key=auth:login:ip; attempts > N	tooMany(key) == true; further login 429 Too Many Requests

TABLE VI: Unit Tests: Authentication & Tokens

#	Component	Inputs	Expected Output
M1	MessagePolicy.canStartConversation	actorRole=Employer; ownsListing=true; applicantId=A1; jobId=J1	true
M2	MessagePolicy.canStartConversation (deny)	actorRole=Applicant; triesToStart=true	false
M3	MessageService.startConversation	employerId=E1; applicantId=A1; jobId=J1; firstMessage=”Hi”	Conversation created/reused; first message persisted; notifier.notify(applicant)
M4	MessageService.startConversation (policy deny)	actorRoleEmployer OR ownsListing=false	403 Forbidden; nothing persisted
M5	MessageService.sendMessage (ok)	conversationId=C1 (E1A1); senderId E1,A1; body=”hello”; files=[]	Body sanitized; attachments validated; message saved; gateway.broadcast; notifier.notify(receiver)
M6	MessageService.sendMessage (non-member)	conversationId=C1; senderId=U9 (not participant)	403 Forbidden; no persist; no notify
M7	ContentFilter.sanitize	body=”script¿alert(1)/script¿ hi”	XSS stripped or escaped; output ” hi”
M8	validateAttachments	files=[image.png(1MB), tool.exe(200KB)]	Reject .exe; return validation error
M9	Rate limiting on send	senderId=E1; sendsPerSec > N	429 Too Many Requests; message not saved/broadcast
M10	Delivery events	messageId=M1; actor=participant/non-participant	markDelivered/markRead update timestamps (participants only); non-participant 403

TABLE VII: Unit Tests: Secure Message Sending

G. Backend: JobListingService

#	Tested Function	Inputs	Expected Output
1	createJob	Valid employerID value and JobListing object	Add the JobListing to the list of jobs (True)
2	createJob	Invalid employerID value (Ex. an ID that is negative, or of the incorrect format or length) or JobListing object (Ex. it has a duplicate jobID to an existing job, has conflicting dates such as an expiry date earlier than the date posted, or if it has empty fields or a negative salary value)	Does not add the JobListing to the list of jobs, instead outputs an error messaged based on what part of the input was invalid (False)
3	getJobById	Valid jobID	Returns the JobListing corresponding to the inputted jobID
4	getJobById	Invalid jobID (Ex. jobID is empty, negative, does not exist in the database or is of the incorrect format, such as being too long or short)	Outputs an error message based on what part of the input was invalid and returns Null
5	getJobsByEmployer	Valid EmployerID	Returns all the jobs listed by an employer in the form of a list of the JobListing class
6	getJobsByEmployer	Invalid EmployerID (Ex. employerID is empty, negative, does not exist in the database or is of the incorrect format, such as being too long or short)	Outputs an error message based on what part of the input was invalid and returns Null
7	updateJob	Details to be updated of a job	A change in the relevant details that are changed to the inputted values. True if the inputs are valid and False if not
8	deleteJob	Valid jobID	Attempt to delete the job. Return True if it was deleted and False if it was not.
9	deleteJob	Invalid jobID (Ex. jobID is empty, negative, does not exist in the database or is of the incorrect format, such as being too long or short)	Outputs an error messaged based on what part of the input was invalid and returns False
10	getJobApplications	Valid jobID and sortBy inputs	It returns a list of the applications corresponding to the provided jobID sorted according to the received sortBy input
11	getJobApplications	Invalid jobID (Ex. jobID is empty, negative, does not exist in the database or is of the incorrect format, such as being too long or short) or sortBy inputs (Ex. sortBy is empty or does not match a preset keyword (such as “ascending date applied”, “descending applicantID” or “status”) that can be used to sort the applications)	Outputs an error message based on what part of the input was invalid and returns Null

TABLE VIII: Backend Unit Tests: JobListingService

H. Help Bot

#	Component	Inputs	Expected Output
1	HelpBot.processQuery	Job seeker U1 asks ”How do I apply for jobs?”	Returns helpful response about swiping right with 95% confidence, no human support needed
2	FAQDatabase.searchFAQs	Job seeker searches for ”upload CV” help	Finds 3 relevant FAQ entries about CV upload, best match has 92% relevance score
3	HelpBot.generateResponse	Employer needs profile help, has profile FAQ data available	Generates personalized message: ”To update your company profile, go to Settings...”
4	HelpBot.escalateToHuman	User U1 reports ”Database error 500”, provides email contact	Creates support ticket T123, notifies human support team, returns success confirmation
5	FAQDatabase.addFAQ	Admin adds ”How to delete account?” FAQ for job seekers with answer	Assigns unique ID FAQ456, adds to searchable database, indexing successful
6	ChatInterface.renderChatWindow	Job seeker U1 opens chat with previous conversation history	Displays chat window with message history, input field ready for typing
7	ChatInterface.sendMessage	User sends ”Help with profile” in conversation C1	Message passes validation, gets timestamped, delivers successfully, triggers bot response
8	HelpBot.updateConversationHistory	Log password reset conversation for user U1 with response	Saves conversation to database with ID LOG789, logging successful
9	FAQDatabase.getFAQsByCategory	Request all FAQs in ”applications” category	Returns 5 job application FAQs, filtered by category and sorted by popularity

TABLE IX: Unit Tests: AI Help Bot

VI. Backlogs

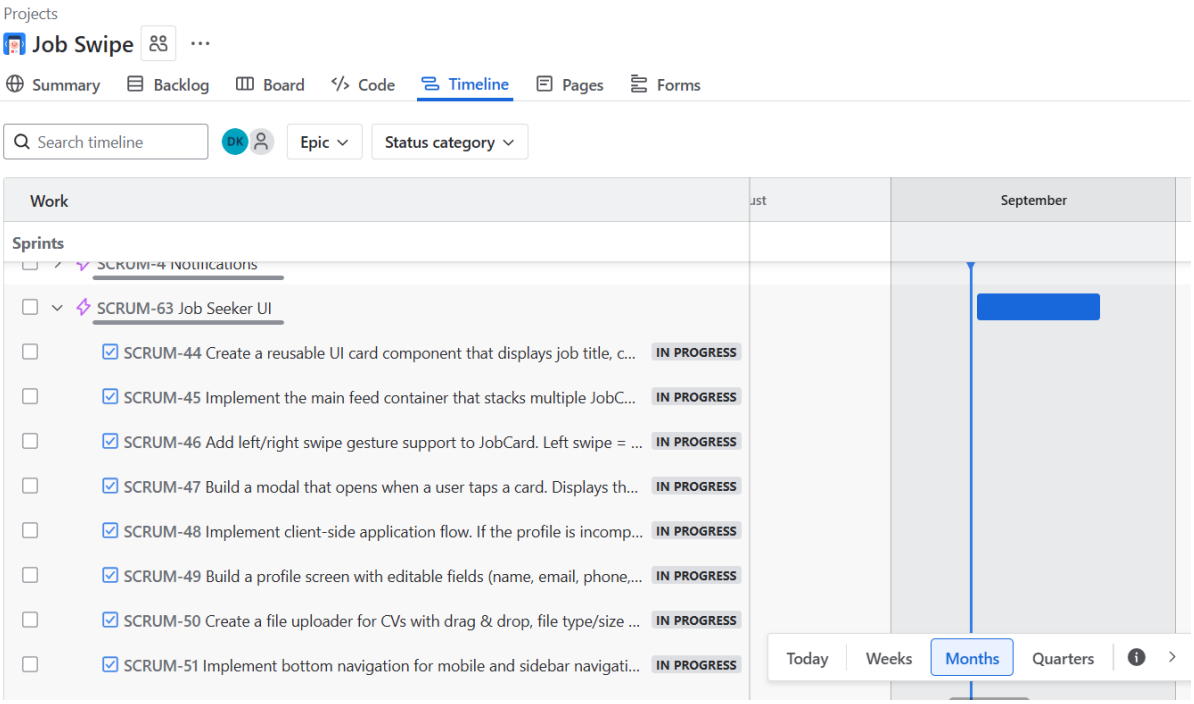


Figure 16: Frontend Job Seeker UI Backlog

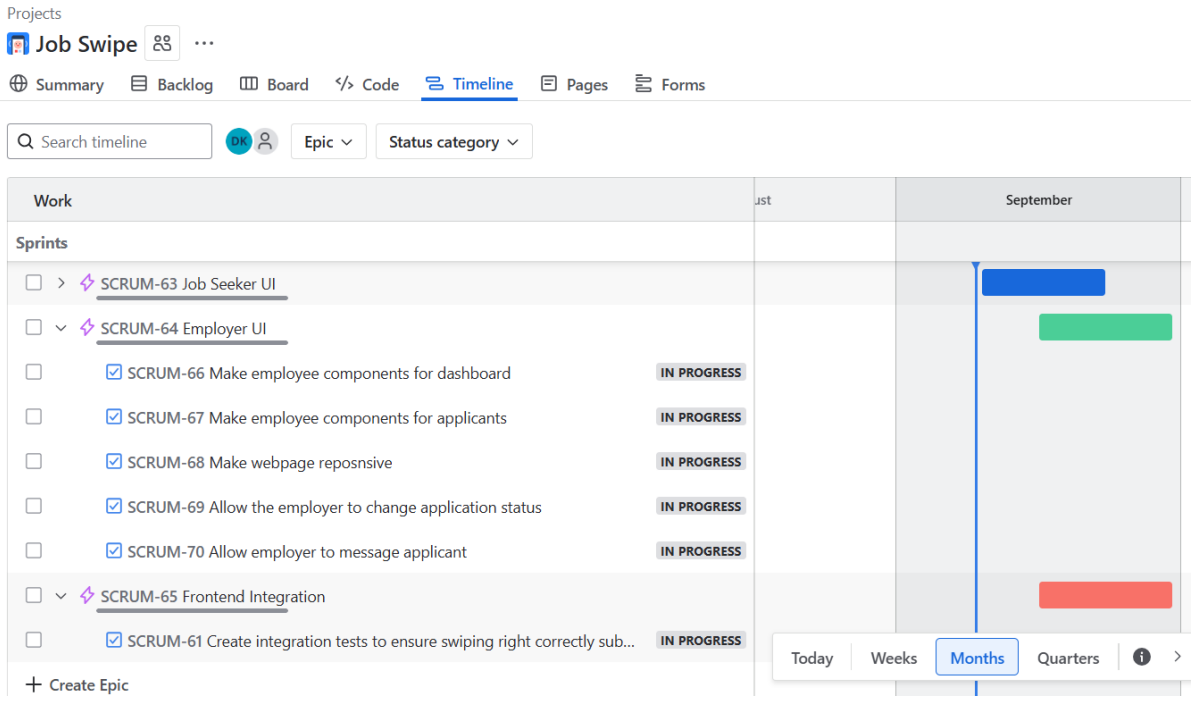


Figure 17: Frontend Employer UI Backlog

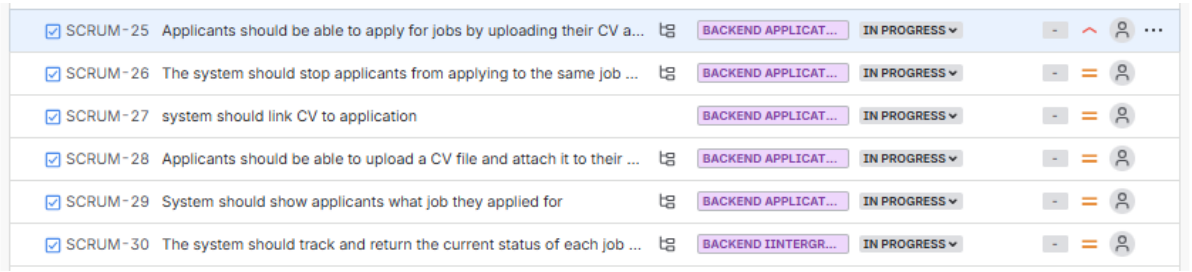


Figure 18: Backend Application Backlog

Figure 19: Backend Employer Integration Jira Backlog

Figure 20: Backend Job Listing Integration Jira Backlog

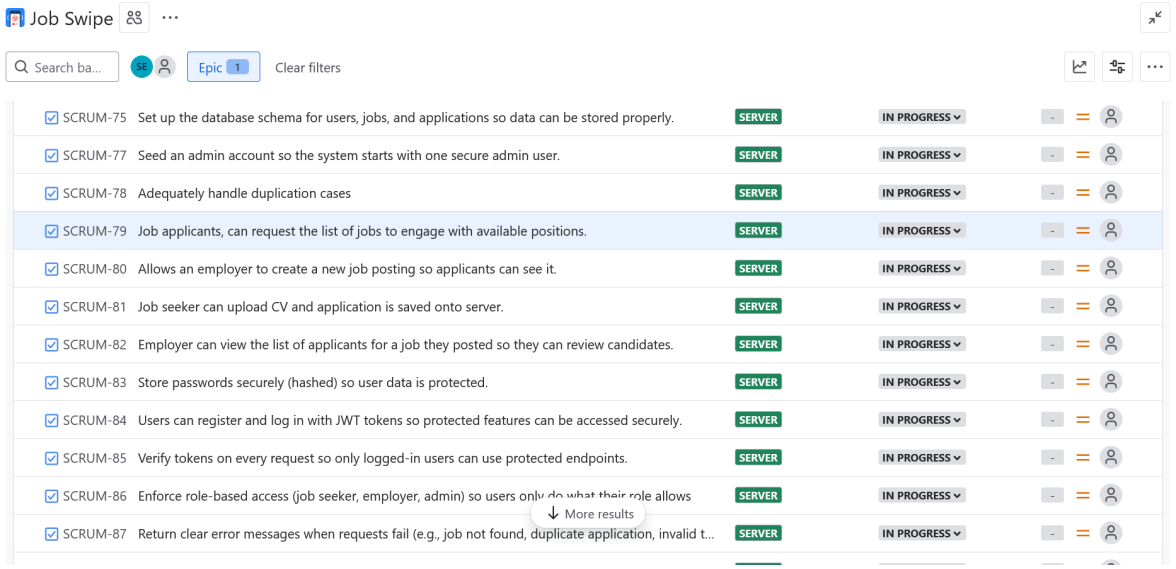


Figure 21: Server Backlog from Jira

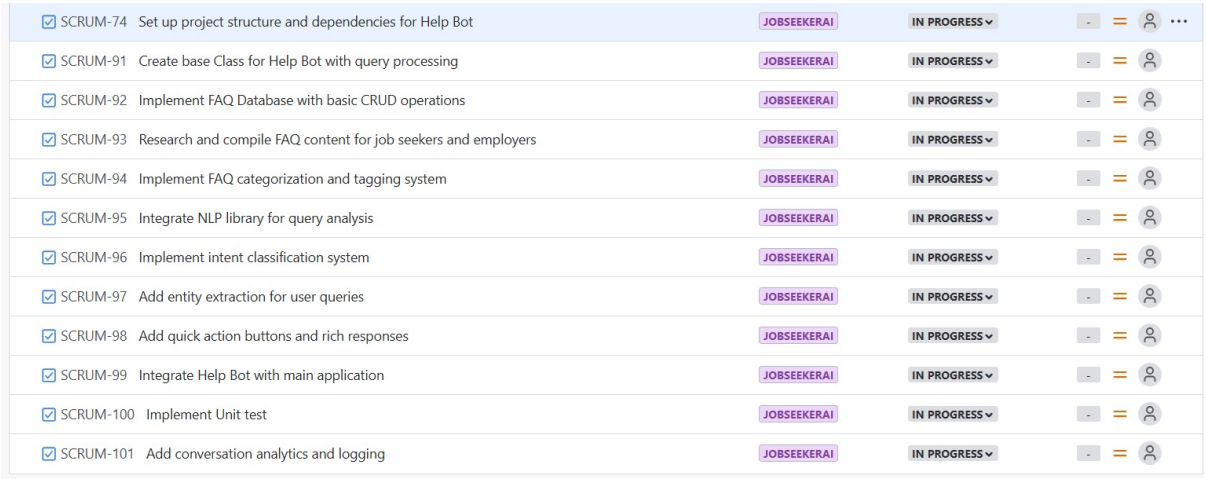


Figure 22: AI Help Bot Backlog from Jira

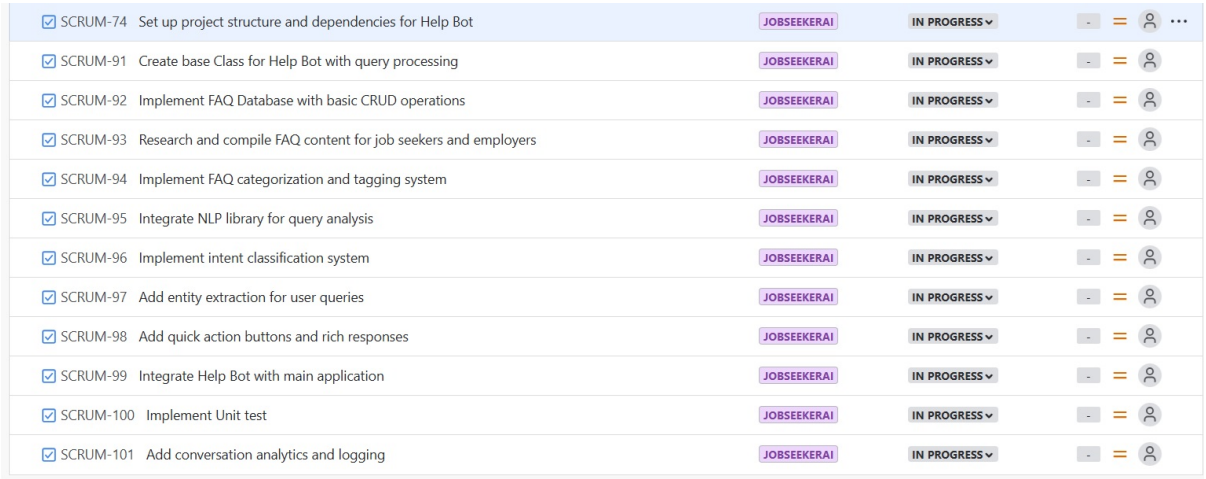


Figure 23: AI Help Bot Backlog from Jira

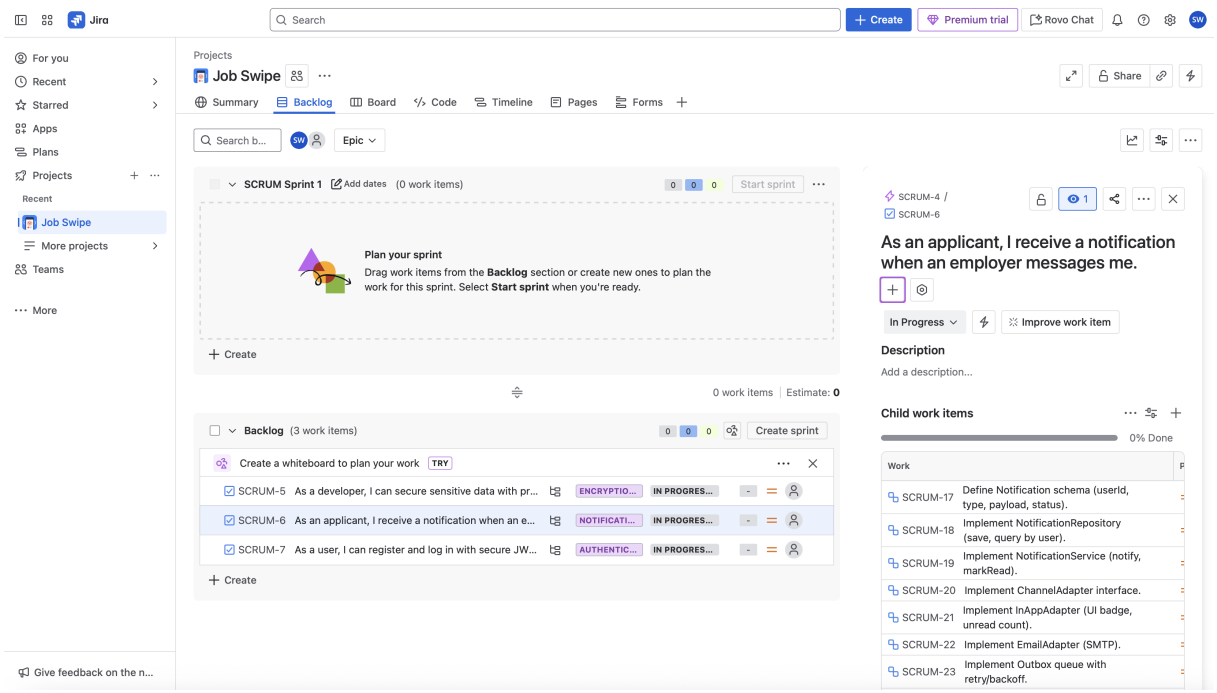


Figure 24: Jira backlog item — Notification Epic

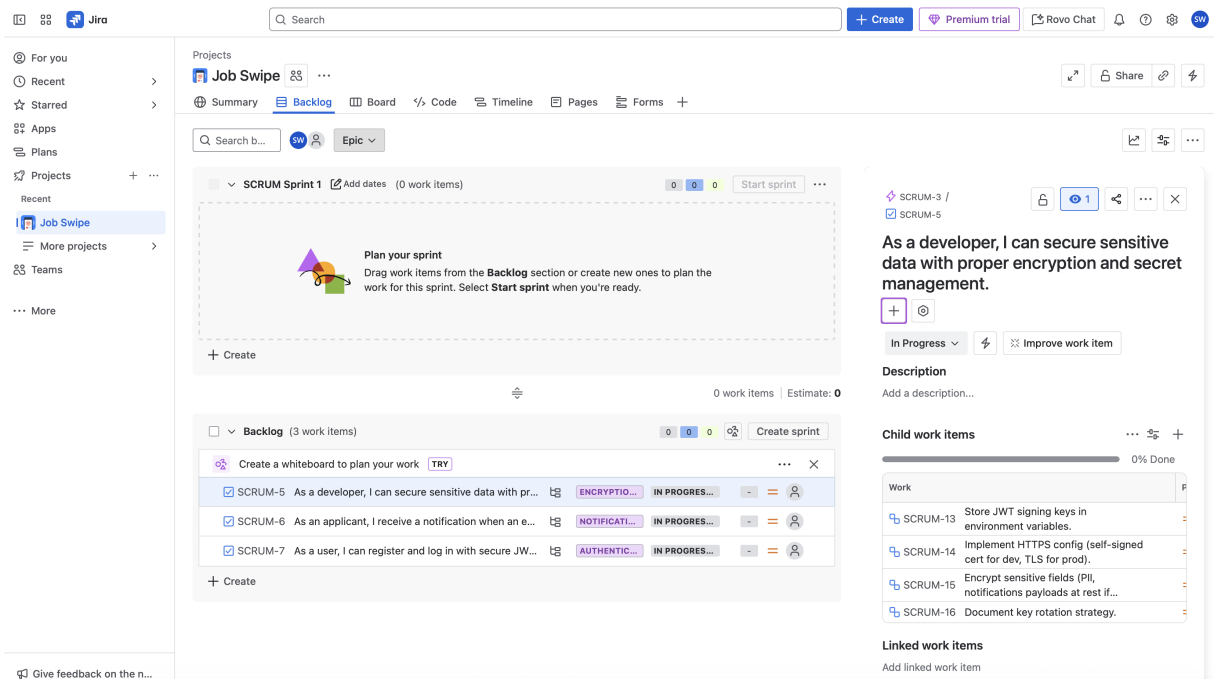


Figure 25: Jira backlog item — Encryption Epic

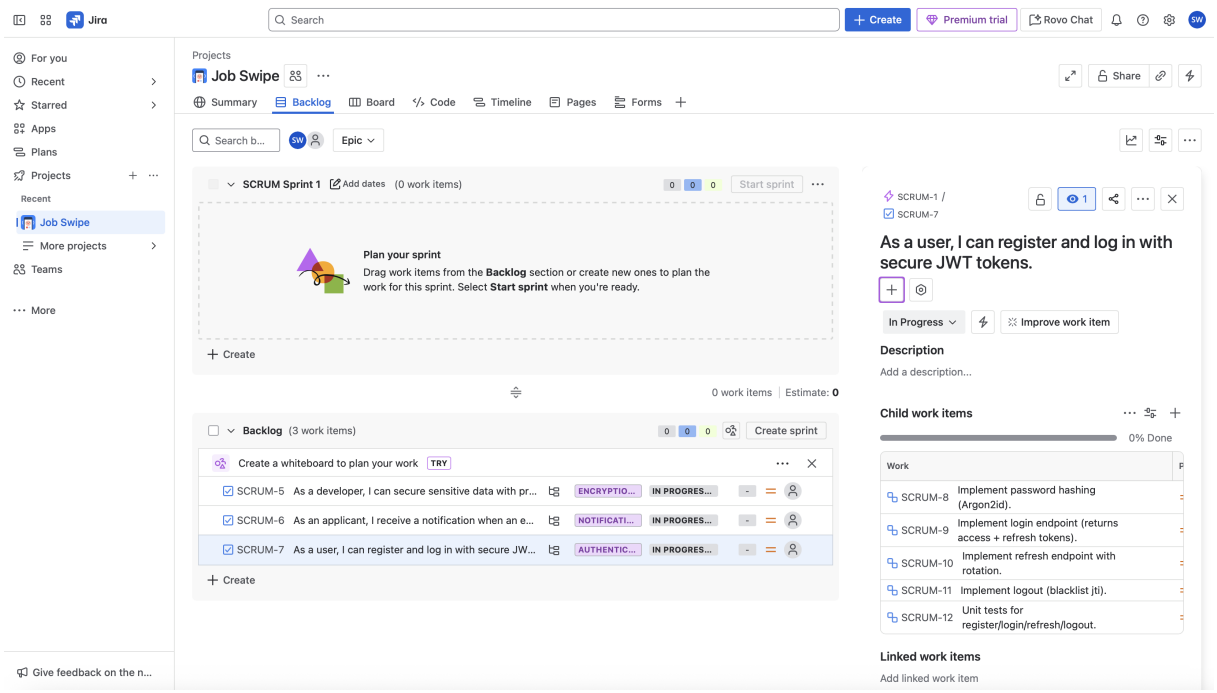


Figure 26: Jira backlog item — Authentication Epic

VII. High Level GUI

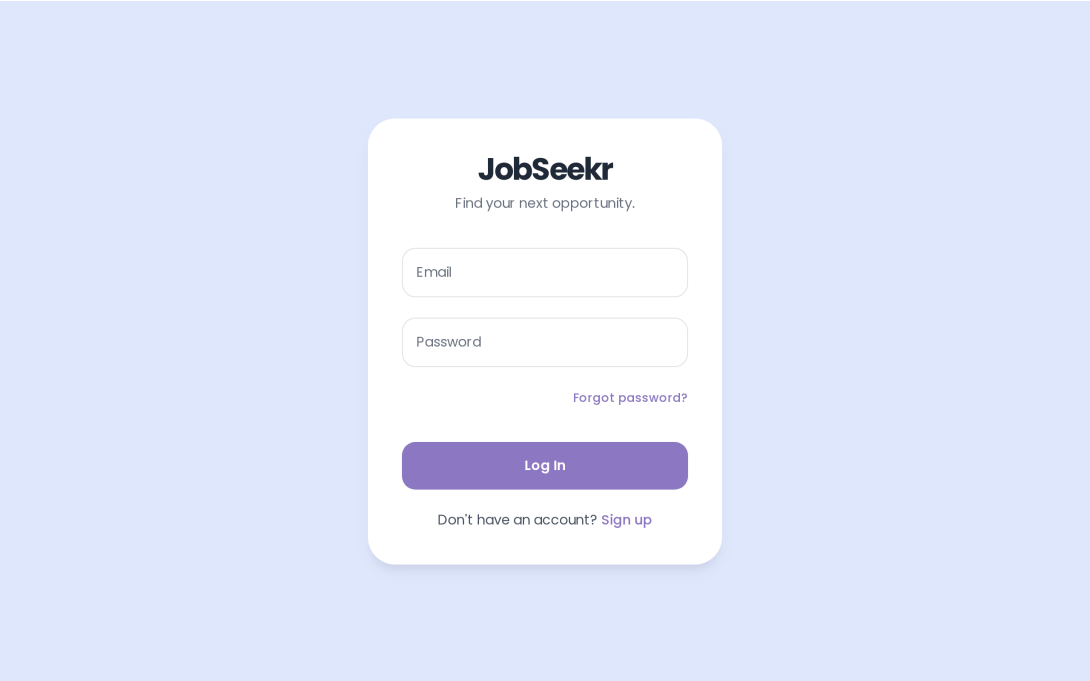


Figure 27: User Login Page

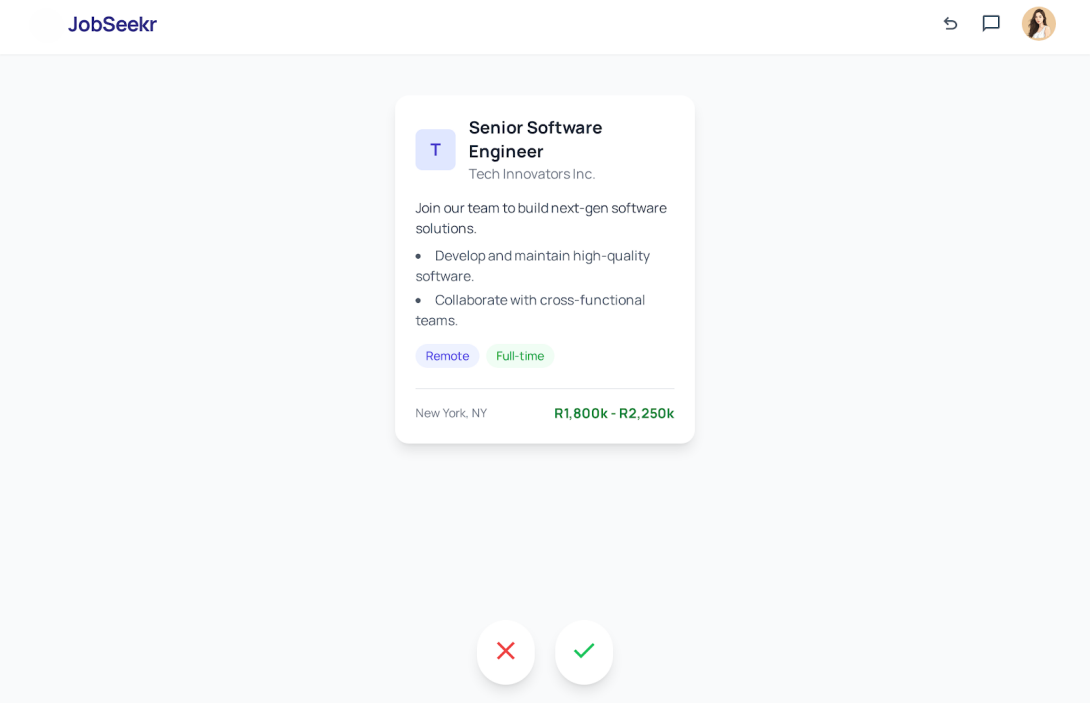


Figure 28: Job Seekers Feed

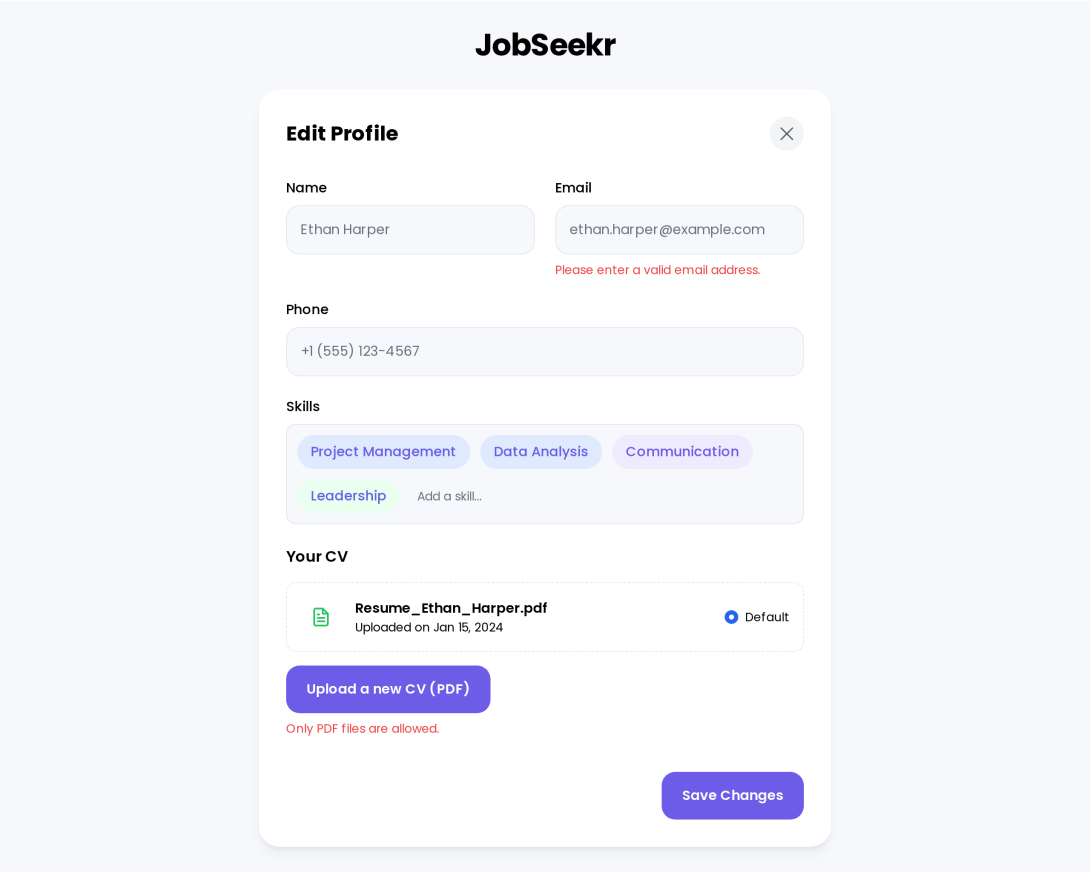


Figure 29: Job Seekers Profile Page

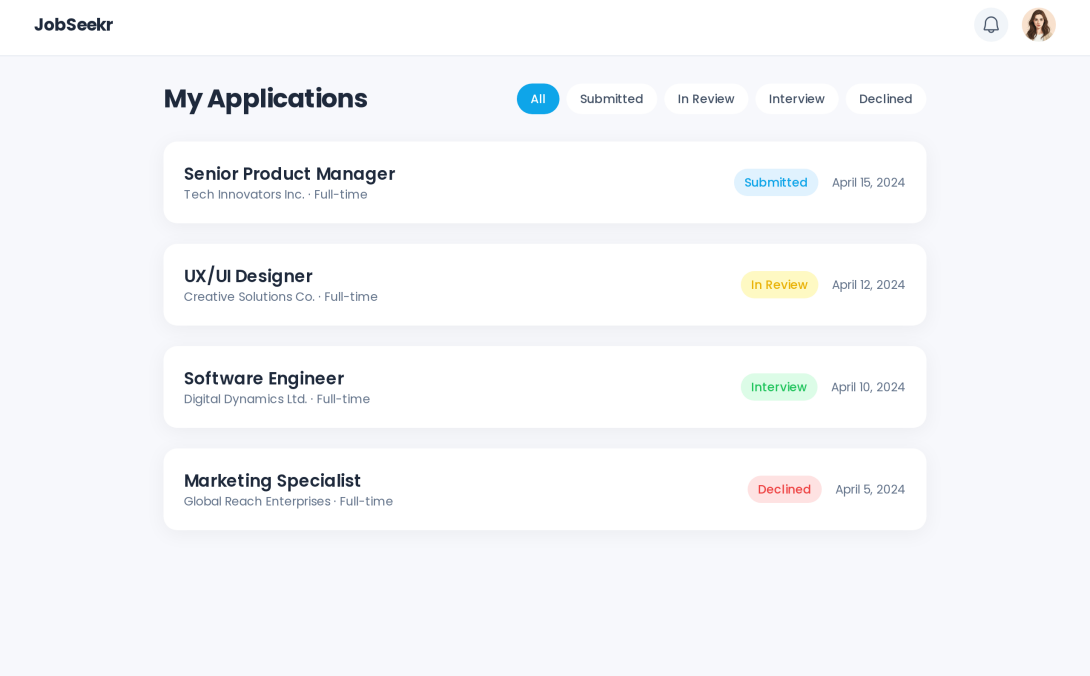


Figure 30: Job Seekers Application History

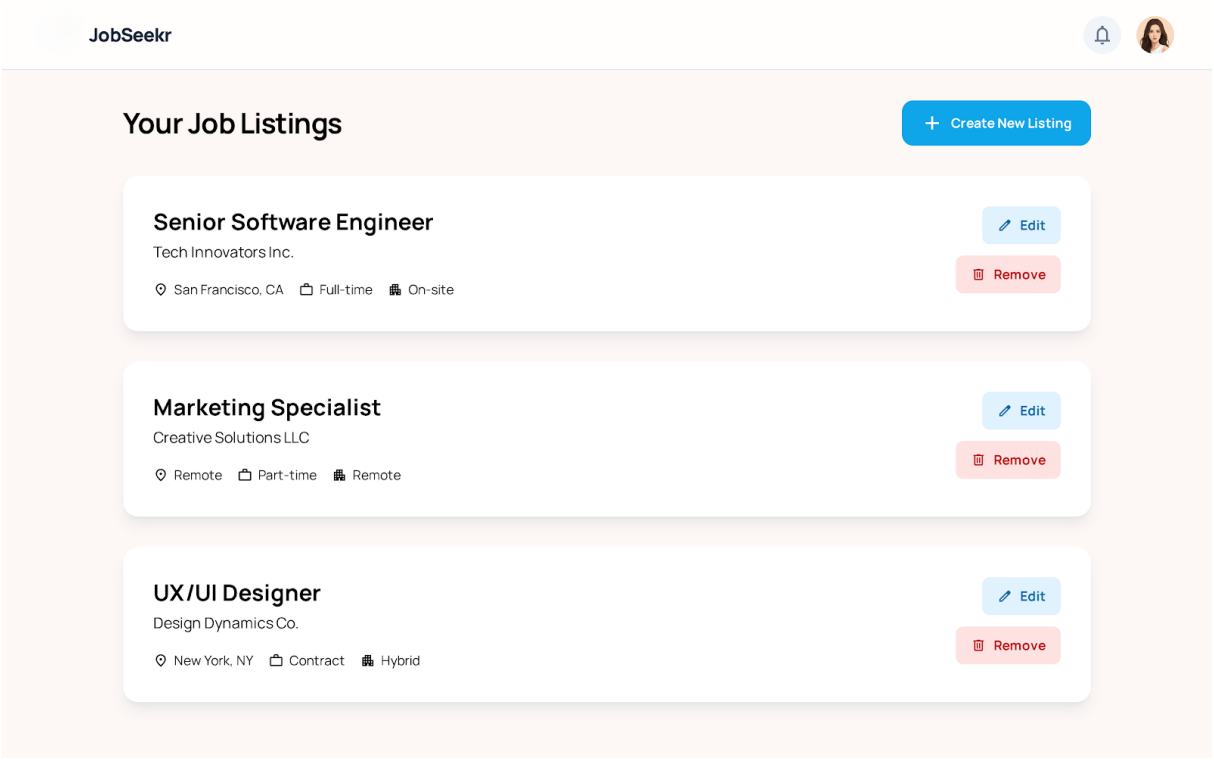


Figure 31: Employers Dashboard

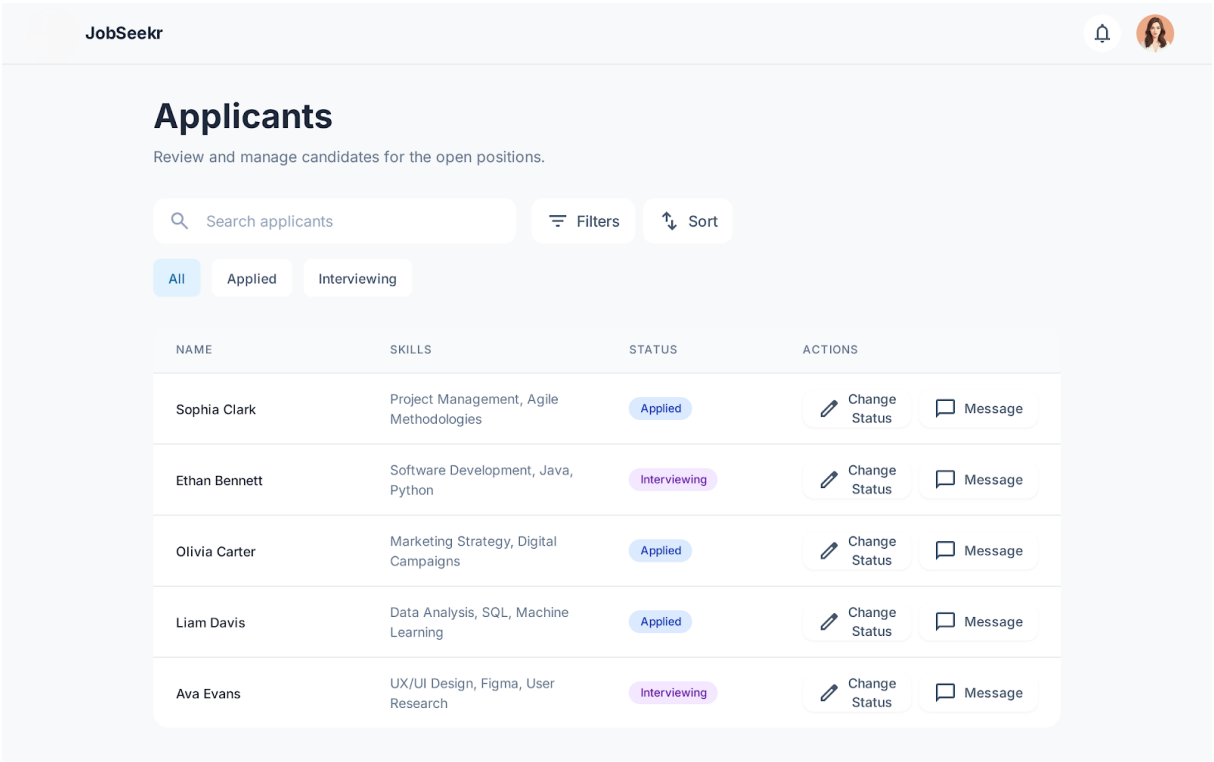


Figure 32: Employers Applicants View